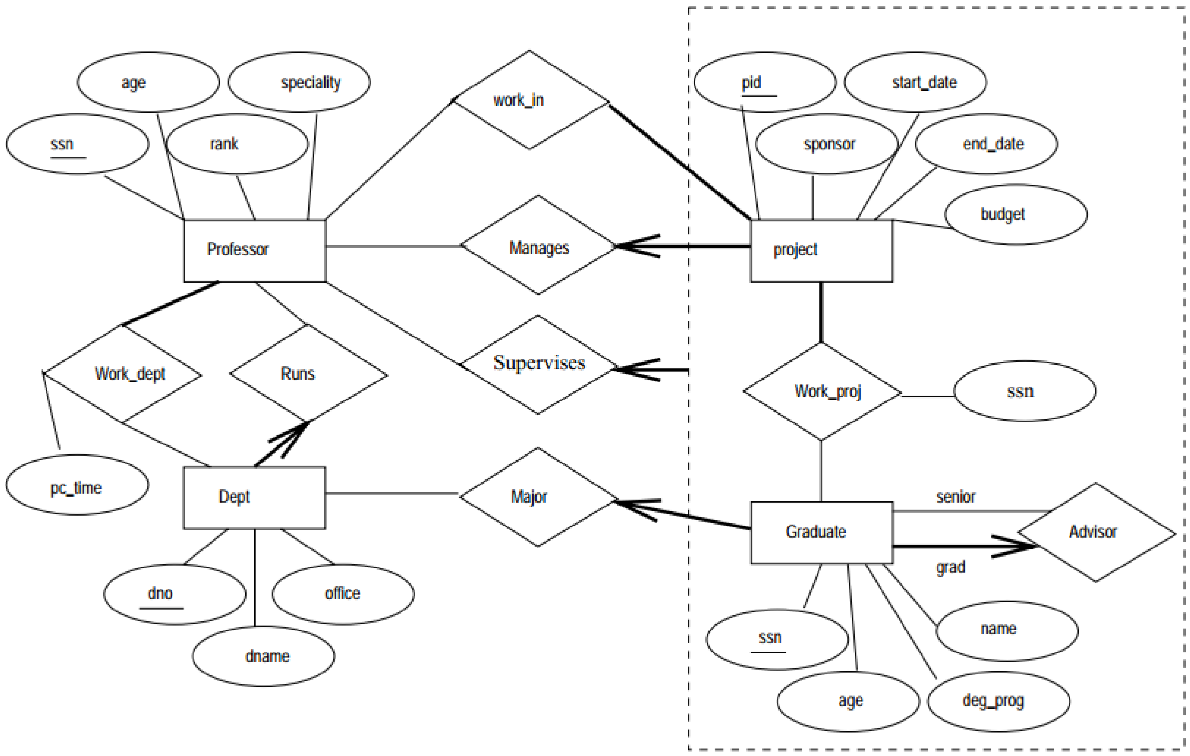
1 Conceptual Design

Question 1(a): Provide a conceptual database model in the form of an ER Diagram. Explain the most important design choices and document relevant assumptions.

Answer:



(The diagram contains two typos: Professor should have an attribute name and work_proj should not have attribute ssn)

Instead of aggregation, one could invent GraduateProject as an entity that refers to one (research) project and one graduate student. This entity has a supervisor attribute. It leads to the same relational model as below.

Question 1(b): Give the associated relational schema, indicating relations R(A1,..) with their attributes Ai, primary keys underlined, and foreign key relationships (→). Separately also comment on NULLable attributes (if any), and constraints that a relational database could check, including candidate keys.

Answer:

professor(<u>psn</u>, age,rank,speciality)
worksForDept(<u>psn</u>→professor,<u>dno</u>→department,workPerc)
department(<u>dno</u>,name,office,director→professor)
project(<u>pid</u>,sponsor,start_date,end_date,budget,manager→professor)
investigator(<u>psn</u>→<u>professor</u>,<u>pid</u>→project)
graduate(<u>gsn</u>,name,age,degreeProgram,advisedBy→graduate, major →department)
worksOnProject(<u>gsn</u>→<u>graduate</u>,<u>pid</u>→<u>project</u>,supervisor→professor)

2 Normalization

Given *R*(*A,B,C,D,E*), and FDs: *f* = { A→DB, B→D, AE→ED, E→A }
Show your intermediate steps in all the answers.
Question 2(a): is *f* canonical (a minimal basis)? If not, make it so.
Question 2(b): what are the minimal (candidate) keys?
Question 2(c): is *R* in BCNF already? If not, make it so, and tell if any FD's get lost.
Question 2(d): is *R* in 3NF already? If not, make it so.

Answer:

(a) The canonical FD algorithm would split A→DB into A→D and A→B and AE→ED into AE→E, AE→D, of which AE→E is trivial and gets deleted.
Left-hand side elimination in the remaining FDs is relevant for AE→D. Here, A can indeed be dropped because E→A ,so AE→D becomes E→D.
We now have {A→B,A→D,B→D,E→A,E→D}.
The final question is if any FD can be dropped,. Now, A→D can be dropped because A→B→D, and also E→D can be dropped because E→A→B→D. So a (the) canonical set of FDs is *f* = { A→B, B→D, E→A}.

(b) Using the optimized version of the candidate key algorithm, we note that C and E are not on a right hand side in the canonical set, so CE must be part of any key. In fact, since CE^+=ABCDE, it is a key. Hence CE is the only minimal key.

(c) R is not in BCNF because none of the lhs is a key (=contains CE).

We first maximize the right hand sides of the canonical set of FDs into *f*" = { A→BD, B→D, E→ABD }
Then, we examine these FDs one by one and split.

No matter what order is applied for splits, you get:
R1(<u>C</u>,<u>E</u>), R2(<u>E</u>,A), R3(<u>A</u>,B), R4(<u>B</u>,D)
No FDs were lost in this decomposition.

(d) this relation is not in 3NF because none of the lhs of the FDs is a key; and in fact all FDs violate 3NF because none of them has a rhs that only consists of keys (CE).

3NF decomposition follows from the canonical FD set:
R1(<u>E</u>,A), R2(<u>A</u>,B), R3(<u>B</u>,D), R4(<u>C</u>,<u>E</u>)
R4 was added because none of R1-R3 contains a key. Note that in this case, BCNF and 3NF produce the same result.

3 SQL

Consider the following schema:

Suppliers(<u>sid</u>,sname,saddress)
Parts(<u>pid</u>,pname,color)
Catalog(<u>sid</u>→Suppliers,<u>pid</u>→Parts,cost)

The primary keys are underlined and → Rel indicates a foreign key relationship with the primary key of Rel.

Formulate the following queries in SQL. You will only obtain maximal points for your answers if your answers avoid use of GROUP BY in favor of existential quantification.

Question 3(a): Find the pids of parts that are supplied by only one supplier

Answer:

```
SELECT C.pid
FROM Catalog C
WHERE NOT EXISTS
    (SELECT C1.sid
     FROM Catalog C1
     WHERE C1.pid = C.pid AND C1.sid <> C.sid)
```

Question 3(b): Find supplier names who sell all black parts

Answer:

```
SELECT S.name
FROM Suppliers S
WHERE NOT EXISTS
    (SELECT "foo"
     FROM Part P
     WHERE P.color = 'black' AND NOT EXISTS
         (SELECT "bar"
          FROM Catalog C
          WHERE C.sid = S.sid AND C.pid=P.pid))
```

4 Transactions

Question 4(a): Describe the notions of a cascadeless schedule and a recoverable schedule.
Answer:

- **Cascadeless schedule**: A schedule S is **cascadeless** if, for every transaction T participating in the schedule, all the read operations in T read values from already committed transactions.
- **Recoverable schedule**: A schedule S is **recoverable** , for every transaction T participating in the schedule, the commit of T comes after the commits of all transactions from which it has read values.

Question 4(b): describe the most efficient way of deciding whether a schedule is conflict serializable. Use this method on the below schedule:

T1:			**R(Z)**		**R(Y)**	
T2:		**R(Y)**		**W(Y) R(V)**		
T3:	**W(V)**					**W(Z)**

Answer: The most efficient method is to create a precedence graph and test if it has a cycle:

due to T1:R(Z) and later T3:W(Z) we have T1→T3
due to T2:W(Y) and later T1:R(Y) we have T2→T1
due to T3:W(V) and later T2:W(Z) we have T3→T2
Since we have a cycle T1→T3→T2→T1, the schedule is not conflict serializable.

5 Database APIs

1.

Question 5(a): discuss the advantages and/or disadvantages of database APIs where the application (e.g. written in Java) assembles a database query as a string, substituting in possible parameters on-the-fly.
Answer:
Advantage: since the query is formulated on the fly, this is flexible (the shape of the query can be varied depending on the parameters), which can be handy sometimes.
Disadvantages: formulating the query as a string leaves one vulnerable for SQL injection. Further, if performance is a concern, it would be better to use prepared statements (the parameters would then be sent separately), potentially also avoiding the SQL injection vulnerability.

Question 5(b): the original ANSI SPARC architecture identified three levels of database modeling; please name them. At which of these levels (highest, middle, lowest) can we see object-relational mappings (ORMs) such as Hibernate and the Entity Framework. Discuss the purpose (function) of this level.
Answer:
Conceptual, Logical and Physical. ORMs can be seen as providing the Conceptual level. It can be used to offer an application its own view of the data. This also allows to evolve the

database schema over time without breaking existing applications (provided that after changing the database schema, one edits the ORM mapping, such as to hide the changes).